

Scalable influence blocking maximization in social networks under competitive independent cascade models



Peng Wu^{a,b}, Li Pan^{a,b,*}

^a School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, 800 Dong Chuan Rd, Shanghai, China

^b National Engineering Laboratory for Information Content Analysis Technology, Shanghai Jiao Tong University, Shanghai, China

ARTICLE INFO

Article history:

Received 21 January 2017

Revised 2 April 2017

Accepted 4 May 2017

Available online 5 May 2017

MSC:

68P10

91D30

Keywords:

Influence blocking maximization

Competitive independent cascade model

Social networks

ABSTRACT

Bad information propagation in online social networks (OSNs) can cause undesirable effects. The opposite good information propagating competitively with bad information can restrain the propagation of bad information. In this paper, we address the Influence Blocking Maximization (IBM) problem aiming to find a set of influential people initiating good information propagation to maximize the blocking effect on the bad information propagation in OSNs. The problem is studied on two competitive propagation models describing competitive propagation processes in two classic situations in OSNs. Two models are derived from the Independent Cascade Model (ICM). Greedy algorithms for IBM problem under two competitive propagation models are slow and not scalable. Thus, we design two heuristics CMIA-H and CMIA-O based on the maximum influence arborescence (MIA) structure to efficiently solve the IBM problem under two competitive propagation models, respectively. Extensive experiments are conducted on real-world and synthetic datasets to compare the proposed algorithms with the greedy algorithms and other baseline heuristics. The results demonstrate that both CMIA-H and CMIA-O achieve matching influence blocking performance to the greedy algorithms and consistently outperform other baseline heuristics, while they are several orders of magnitude faster than the greedy algorithms.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

Online Social Networks (OSNs), such as Facebook, Twitter and Weibo, etc., have become one of the most important platforms for the propagation of news, ideas, opinions, adoption of new products, etc.. Such information propagation is generally referred as influence propagation [1]. While OSNs are very beneficial for genuine and trustworthy information dissemination, their openness also facilitates the propagation of rumors, gossips and other kinds of misinformation which could potentially cause undesirable effects and even lead to serious economic and political consequences [2–5]. For example, the rumor that “Two bombs had exploded at the White House, injuring Barack Obama” from hacked Associated Press Twitter account resulted to 10 billion USD losses [6]. Similarly, the negative information about a product propagating in OSNs can cause huge losses to a company. Misinformation and negative information are collectively called bad information in this paper.

In order to make OSNs a more reliable platform for information dissemination, it is crucial to seek effective strategies to re-

duce the devastating effects of bad information [2,7]. If users have accepted good information used to correct bad information, they will hardly be influenced by such bad information [1,2,7,8]. Thus the propagation of good information can restrain the propagation of corresponding bad information in OSNs [1,2,7–12]. Given a set of bad information sources that initiate bad information propagation in OSNs, we study the Influence Blocking Maximization (IBM) problem of finding a set of users from whom the good information is disseminated so that the blocked negative influence range of bad information is maximized. The problem is studied on two competitive propagation models in this paper, i.e., Multi-Campaign Independent Cascade Model (MCICM) with high-effectiveness property and Campaign-Oblivious Independent Cascade Model (COICM) [2]. These two models are extended from Independent Cascade Model (ICM) [13] which is a classic stochastic propagation model describing how single information is propagated on the network starting from a set of seed nodes [1]. MCICM with high-effectiveness property and COICM describe and model competitive propagation processes in two classic situations, respectively. It has been proved that the IBM problems under above two models are NP-hard and their objective functions are submodular [2]. Thus, greedy algorithms can achieve an approximation ratio of $1 - 1/e - \epsilon$ of the optimal solution, where e is the base of natural logarithm, and ϵ

* Corresponding author.

E-mail addresses: panli@sjtu.edu.cn, catking@sjtu.edu.cn (P. Wu).

depends on the accuracy of the Monte–Carlo estimate of the influence spread range. However, the greedy algorithms are very slow for large networks, if we want to keep the above ϵ small by running the Monte–Carlo simulations a large number of times. Given that OSNs are usually very large and blocking bad information propagation typically requires immediate strategies before it propagates too far, we believe that the scalability issue of the greedy algorithms for IBM problems will hinder their application in containment of bad information propagation in OSNs.

In this paper, we address the scalability issue of the greedy algorithms by designing two efficient heuristic algorithms CMIA-H and CMIA-O for the IBM problem under MCICM with high-effectiveness property and COICM, respectively. The good (bad) information propagation is referred to as positive (negative) propagation and the good (bad) information spread range that measures the number of nodes accepting good (bad) information is referred to as positive (negative) influence spread. The main reason of the inefficiency of the greedy algorithms is that they spend a lot of time to estimate the negative influence spread by a large number of Monte–Carlo simulations. Inspired by [14], we approximately compute the negative influence spread in the local Maximum Influence Arborescence (MIA) structure of every node in two designed algorithms. The influence spread computation in MIA is more than three orders of magnitude faster than that by Monte–Carlo simulations. Thus two designed algorithms can be much faster than greedy algorithms. It should be noted that the influence spread computation procedures here are more complex than that in [14], since our propagation models model two competitive propagations while theirs only model a single propagation. Moreover, the computation procedures under MCICM with high-effectiveness property and COICM are different. Specifically, for MCICM with high-effectiveness property, we adopt the high-effectiveness property of the positive propagation to help compute the blocked negative influence of any node. While for COICM, we design a dynamic programming method to compute the blocked negative influence of any node. Then the node with the largest blocked negative influence is iteratively selected as the new positive seed. To test the efficiency and effectiveness of the proposed CMIA-H and CMIA-O, we conduct extensive experiments on several real-world and synthetic networks. The performance of two proposed algorithms is compared with both the greedy algorithms and several heuristic algorithms, and the results show that (a) compared with the greedy algorithms, CMIA-H and CMIA-O have matched influence blocking performance, while they are several orders of magnitude faster to run; and (b) compared with other scalable heuristics, CMIA-H and CMIA-O consistently outperform them in negative influence blocking performance with a significant margin.

The rest of the paper is organized as follows. Some related works are discussed in Section 2. Section 3 formalizes the IBM problem under the two competitive propagation models, i.e., MCICM with high-effectiveness property and COICM. In Section 4, we describe the CMIA-H algorithm for IBM problem under MCICM with high-effectiveness property, and the CMIA-O algorithm for IBM problem under COICM. The experiments results are analyzed in Section 5. Finally, Section 6 concludes the paper.

2. Related work

The influence maximization (IM) problem is first formulated as a combinatorial optimization problem by Kemp et al. [13]. They study the IM problem under the classic ICM and Linear Threshold Model (LTM), and propose a standard greedy algorithm with approximation guarantee of $1 - 1/e - \epsilon$. LTM is another single information stochastic propagation model. To address the scalability issue of the greedy algorithm, many follow-up studies [14–19] try to propose much more efficient heuristics for the IM problem. The

scalable LDAG algorithm of [16] utilizes the linear computation property of influence in directed acyclic graphs (DAG) to efficiently approximate the influence spread of any seed set under the LTM. The scalable PMIA algorithm of [14] utilizes MIA structure to efficiently approximate the influence spread of any seed set under the ICM. Our CMIA-H algorithm and CMIA-O algorithm are also based on the efficient influence computation property of the MIA structure, but with nontrivial extensions to deal with competitive influence propagation and IBM problem.

Different from previous IM problem that aim at maximizing the influence spread of one propagation, containment of bad information aims at minimizing the negative influence spread by either blocking bad information at influential users [5,20–24], or clarifying bad information by spreading good information [1,2,7–10]. In reality, blocking rumors at influential users may incur criticism since it has risks of violating human rights [3], while spreading good information competing with bad information is more moderate. Most of works about blocking bad information by spreading good information are studied under competitive propagation models extended from either ICM or LTM. Nguyen et al. [7] study the β_T^I -Node Protectors problems which aim to find the smallest set of nodes with good information to contain the spread of misinformation under two models extended from ICM and LTM, respectively. Fan et al. [9] address the Least Cost Rumor Blocking (LCRB) problem which aims at identifying a minimal subset of individuals as initial protectors to ensure bridge ends in neighbor communities of the rumor community protected under a model from ICM. Different from IBM problem studied in this paper, above two works study the seed set minimization problem. He et al. [1] study the IBM problem under the CLTM coming from LTM and propose CLDAG algorithm based on the DAG structure to efficiently solve the problem. Budak et al. [2] study the IBM problems under the MCICM with high-effectiveness property and the COICM, respectively. They prove that the IBM problems under two propagation models are both NP-hard, and their objective functions are submodular. Recently, several NP-hard problems about enhancing the performance of communication networks have been solved by algorithms mimicking biological processes [25,26]. As for problems with submodular objective functions, it has been proved that the greedy algorithms can solve them with approximation guarantee of $1 - 1/e$ [27,28]. However, the greedy algorithms for IBM problems are slow, as they require a large number of Monte–Carlo simulations to estimate the influence spread. Motivated by the extremely low efficiency of the greedy algorithms, this paper proposes two scalable heuristic algorithms CMIA-H and CMIA-O based on MIA structure [14] for the IBM problems under MCICM with high-effectiveness property and COICM, respectively. The CLDAG algorithm [1] inspires us in designing CMIA-O algorithm for COICM, but we propose a novel influence computation procedure. The comparisons of works most related to this paper are listed in Table 1.

3. Propagation models and problem definition

A social network is modeled as a directed graph $G = (V, E)$, where V is a set of n nodes representing individuals and E is a set of m edges representing relationships. Each node is associated with some states indicating whether it accepts the information or not. Propagation is a state cascade process where the state changes of the nodes depend on the state changes of their in-neighbors, and will further influence the state changes of their out-neighbors. The propagation models define some rules to capture such processes on networks. Nodes accepting the information are considered as being influenced by the information. Thus information propagation can be referred to as influence propagation. Two competitive propagation models, the MCICM with high-effectiveness property and the COICM [2], are first described as follows. Let P and N be the

Table 1

Comparisons of related works. Problems: research problems. Models: propagation models. Algorithms: classic algorithms. Local structures: structures used to compute influence spread in algorithms.

Problems	Models	Algorithms	Local structures	Pros	Cons
IM	ICM [13]	Greedy [13]	–	Approximation guarantee	Too slow
		PMIA [14]	MIA [14]	Fast and has matching performance to greedy algorithm	No approximation guarantee
	LTM [13]	Greedy [13]	–	Approximation guarantee	Too slow
		LDAG [16]	DAG [16]	Fast and has matching performance to greedy algorithm	No approximation guarantee
IBM	MCICM with high-effectiveness property [2]	Greedy [2]	–	Approximation guarantee	Too slow
		CMIA-H (this paper)	MIA [14]	Fast and has matching performance to greedy algorithm	No approximation guarantee
	COICM [2]	Greedy [2]	–	Approximation guarantee	Too slow
		CMIA-O (this paper)	MIA [14]	Fast and has matching performance to greedy algorithm	No approximation guarantee
	CLTM [1]	Greedy [1]	–	Approximation guarantee	Too slow
		CLDAG [1]	DAG [16]	Fast and has matching performance to greedy algorithm	No approximation guarantee

symbols for the positive propagation and negative propagation, respectively. Each node is allowed to be in one of the three states, i.e., positively active state, negatively active state and inactive state, and it does not change state once it becomes positively active or negatively active. Being positively (negatively) active means being influenced by good (bad) information. The initial set of positively (negatively) active nodes for propagation P (N) is denoted as S_P (S_N). Each edge (u, v) is associated with two propagation probabilities, positive propagation probability $q_{P, u, v}$ and negative propagation probability $q_{N, u, v}$. The MCICM with high-effectiveness property and the COICM address two classic competitive propagation situations by setting different conditions for propagation probabilities. On the one hand, MCICM with high-effectiveness property [2] describes the propagations of good information and misinformation in networks. Since good information is usually announced by authorities and disseminating it can enhance users' credit, while misinformation often starts from less influential users and disseminating it has moral and legal pressure, the propagation probability of misinformation is usually much smaller than that of good information. Thus in order to simplify the model, MCICM with high-effectiveness property assumes that the propagation probability of positive propagation is 1 on each edge, i.e., $q_{P, u, v} = 1$ if there is an edge from u to v and $q_{P, u, v} = 0$ otherwise. On the other hand, COICM [2] models any two competing propagations with similar propagation tendency by setting the propagation probabilities of two propagations be the same on every edge, i.e., $q_{P, u, v} = q_{N, u, v}$ for every edge. For example, two news articles about the same event may have similar propagation tendency in a social network, and similar for two campaigns trying to get users to adopt similar products. When a node u first becomes positively (negatively) active in step t , it has a single chance to activate each currently inactive neighbor v to be positively (negatively) active with probability $q_{P, u, v}$ ($q_{N, u, v}$) in step $t + 1$. If an inactive node v has multiple newly activated neighbors, neighbors with good information will try to influence v first in an arbitrary order, and if their attempts fail, neighbors with bad information continue to try to influence v . The positive dominance rule makes sense, because good information is usually an authorized announcement to correct the corresponding bad information [7]. We focus on the situations that two competing propagations start at the same time. The diffusion process ends when there is no newly activated node in either propagation.

The IBM problem is defined under any propagation model modeling two competing propagations. We first define the Blocked Set of any positive seed set.

Definition 1 (Blocked Set). Given a graph $G = (V, E)$, propagation probabilities $q_{P, u, v}$ and $q_{N, u, v}$, a propagation model and a negative seed set S_N , the blocked set of a positive seed set S_P , denoted as $B(S_P | q_{P, u, v}, q_{N, u, v})$, is defined as the set of nodes that would be negatively active if there is no positive seed set, but are not negatively active if the positive seed set is S_P .

Intuitively, the blocked set is the set of nodes that have been blocked from negative influence by positive influence [1]. The Blocked Negative Influence of any positive seed set is further defined.

Definition 2 (Blocked Negative Influence). Given the blocked set $B(S_P | q_{P, u, v}, q_{N, u, v})$ of a positive seed set S_P , the blocked negative influence $\sigma(S_P)$ of S_P is defined as the expected size of $B(S_P | q_{P, u, v}, q_{N, u, v})$, with expectation taken over $q_{P, u, v}$ and $q_{N, u, v}$, i.e.,

$$\sigma(S_P) = E_{q_{P, u, v}, q_{N, u, v}}(|B(S_P | q_{P, u, v}, q_{N, u, v})|).$$

Now we can define the IBM problem.

Definition 3 (IBM problem). Given a graph $G = (V, E)$, propagation probabilities $q_{P, u, v}$ and $q_{N, u, v}$, a propagation model, a negative seed set S_N , and a positive integer k , the IBM problem aims to find the optimal positive seed set S_P^* of size at most k to initiate positive propagation competing with negative propagation so that the blocked negative influence $\sigma(S_P)$ is maximized, i.e.,

$$S_P^* = \arg \max_{|S_P| \leq k} \sigma(S_P).$$

The IBM problem is a combinatorial optimization problem whose objective function is the blocked negative influence. It has been proved in [2] that the IBM problems under both MCICM with high-effectiveness property and COICM are NP-hard, and their objective functions are monotone and submodular [2]. A submodular function is a set function with a “diminishing returns” property that the incremental value of the function from adding an element to a set S is at least as high as the incremental value from adding the same element to a superset of S [13]. Formally, a submodular function f satisfies

$$f(S \cup \{v\}) - f(S) \geq f(T \cup \{v\}) - f(T),$$

for any element $v \notin T$ and any pair of sets $S \subseteq T$. The greedy algorithm can optimize the submodular function with approximation guarantee of $1 - 1/e$ [27,28]. By adopting Monte-Carlo simulations to estimate the objective function value σ , the following greedy algorithm approximates the optimum of the IBM problem within a factor of $1 - 1/e - \epsilon$: start with the empty positive seed set S_P ,

and iteratively select node that provides the largest marginal gain of σ into S_p until k nodes are selected. That estimating σ under either MCICM with high-effectiveness property or COICM requires a large number of Monte-Carlo simulations to keep the ϵ small makes the greedy algorithm quite slow. We tackle such efficiency problem by proposing two heuristic algorithms CMIA-H and CMIA-O for MCICM with high-effectiveness property and COICM, respectively.

4. Algorithms

Both CMIA-H and CMIA-O have the similar framework with the greedy algorithm that iteratively selects node that provides the largest marginal gain of blocked negative influence into the positive seed set until k nodes are selected. However, they compute the blocked negative influence and its marginal gain approximately in the local MIA structure [14] rather than by Monte-Carlo simulations. MIA structure is originally proposed to estimate influence spread in the ICM. MIA is a joint name of Maximum Influence In-Arborescence (MIIA) and Maximum Influence Out-Arborescence (MIOA). The influence from one node to another is assumed to only propagate on the maximum influence path that has the maximum propagation probability. The MIIA (MIOA) of a node u , denoted as $MIIA(u, \theta)$ ($MIOA(u, \theta)$), is the union of the maximum influence paths to (from) u whose propagation probabilities are equal to or greater than an influence threshold θ . After translating the propagation probability $q_{u,v}$ of each edge (u, v) to a distance weight $-\log q_{u,v}$, the maximum influence path from one node u to another node v is simply the path with minimum distance weight from u to v in the weighted graph. Thus the MIIA and MIOA can be efficiently constructed by algorithms such as Dijkstra algorithm. Let $N^{in}(v)$ denote the set of in-neighbors of v in $MIIA(u, \theta)$. Given a seed set S for ICM, the activation probability of any node v in $MIIA(u, \theta)$, denoted as $ap(v, S, MIIA(u, \theta))$, can be computed recursively by an algorithm [14] in Algorithm 1. For expres-

Algorithm 1 $ap(v, S, MIIA(u, \theta))$.

```

1: if  $v \in S$  then
2:    $ap(v) = 1$ ;
3: else if  $N^{in}(v) = \emptyset$  then
4:    $ap(v) = 0$ ;
5: else
6:    $ap(v) = 1 - \prod_{w \in N^{in}(v)} (1 - ap(w) \cdot q_{w,v})$ ;
7: end if
```

sion simplicity, $MIIA(u, \theta)$ and S are dropped from some notations in Algorithm 1. For more details about MIA structure and influence computation in MIA under ICM, refer to [14].

Different from classic ICM, positive and negative influence propagate concurrently and interfere with each other under both MCICM with high-effectiveness property and COICM. Moreover, for the IBM problem, the blocked negative influence of any positive seed set S_p rather than influence itself is required. Thus, the influence computation procedures in MIA should be redesigned under MCICM with high-effectiveness property and COICM, respectively. In the next two subsections, we describe the CMIA-H algorithm for IBM problem under MCICM with high-effectiveness property, and the CMIA-O algorithm for IBM problem under COICM, respectively. We first list some initials used in the designs of two algorithms in Table 2 for the convenience of readers.

4.1. CMIA-H algorithm for MCICM with high-effectiveness property

In MCICM with high-effectiveness property, we construct MIIA and MIOA for negative propagation by using negative propagation

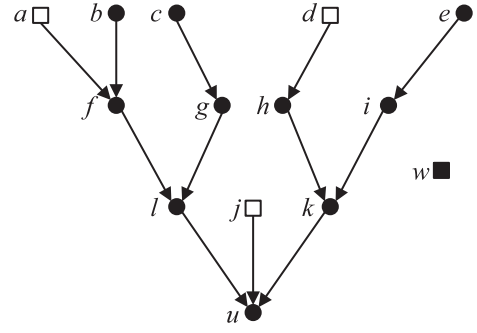


Fig. 1. A toy example of $MIIA(u, \theta)$. a, d, j are negative seeds, and w is a positive seed.

probability. Since the positive propagation probability of any path is 1, the MIIA and MIOA for positive propagation make no sense for influence computation. The positive influence from one node to another propagate on the path with minimum number of hops, i.e., the shortest path. Thus, in this subsection $MIIA(v, \theta)$ and $MIOA(u, \theta)$ represent MIIA and MIOA for negative propagation, respectively. Let $d_p(u, v)$ denote the number of hops, i.e., length, of shortest path from u to v in original graph, and $d_N(u, v)$ denote length of path from u to v in $MIIA(v, \theta)$. Suppose there is a negative seed v and a positive seed w , and v is in $MIIA(u, \theta)$. Since the positive propagation probability of the shortest path is 1, the negative influence from v to u in $MIIA(u, \theta)$ will be totally blocked by w if $d_p(w, u) \leq d_N(v, u)$. On the other hand, if $d_p(w, u) > d_N(v, u)$, the negative influence from v to u in $MIIA(u, \theta)$ will not be obstructed by w at all. In other words, the blocked negative influence of a positive seed depends on the length of shortest path from it to any node influenced by any negative seed. A positive seed can protect a node u from negative influence of any negative seed whose path to u in $MIIA(u, \theta)$ is longer than the shortest path from such positive seed to u . For example, the $MIIA(u, \theta)$ of a node u is illustrated in Fig. 1. a, d, j are negative seeds in $MIIA(u, \theta)$ and $d_N(j, u) = 1$, $d_N(a, u) = 3$, $d_N(d, u) = 3$. Assume there is a positive seed w . If $d_p(w, u) \leq 1$, negative influence from all three negative seeds to u will be blocked. If $1 < d_p(w, u) \leq 3$, negative influence from j will not be blocked, but negative influence from a and d will be blocked. If $d_p(w, u) > 3$, negative influence from all three negative seeds will not be blocked. Let $MIIA(u, \theta, d)$ be a subset of $MIIA(u, \theta)$ which consists of all nodes v in $MIIA(u, \theta)$ with $d_N(v, u) \leq d$. Let $d_c(u, S_p)$ denote the length of shortest path from the closest positive seed in S_p to u ($d_c(u, S_p)$ is set as the positive infinity if there is no path from positive seeds in S_p to u). Then only negative seeds in $MIIA(u, \theta, d_c(u, S_p) - 1)$ have negative influence on u . Thus the negative activation probability of u , i.e., $ap^N(u, S_N, MIIA(u, \theta))$, should be computed in $MIIA(u, \theta, d_c(u, S_p) - 1)$ by Algorithm 1. Accordingly, the incremental blocked negative influence of a potential positive seed w from any node u after adding w to the positive seed set S_p , $DecInf(w, u, S_p)$, can be computed as

$$DecInf(w, u, S_p) = ap^N(u, S_N, MIIA(u, \theta, d_c(u, S_p) - 1)) - ap^N(u, S_N, MIIA(u, \theta, d_c(u, S_p \cup \{w\}) - 1)).$$

Then the total blocked negative influence of w is

$$DecInf(w, S_p) = \sum_{u \in V \setminus (S_p \cup S_N)} DecInf(w, u, S_p)$$

Take toy example in Fig. 1 to illustrate above property. Assume that $d_c(u, S_p) = 2$, then $MIIA(u, \theta, 1)$ consists of nodes j, k, l, u , and only negative seed j has negative influence on u . If $d_p(w, u) = 1$, then $d_c(u, S_p \cup \{w\}) = 1$, and $DecInf(w, u, S_p) = ap^N(u, S_N, MIIA(u, \theta, 1)) - ap^N(u, S_N, MIIA(u, \theta, 0)) = q_{N,j,u}$. In or-

Table 2

List of initials used in the designs of two algorithms.

Propagation models	Algorithms	Local structures for positive propagation	Local structures for negative propagation
MCICM with high-effectiveness property	CMIA-H	PIIS (Positive Influence In Set), PIOS(Positive Influence Out Set)	MIIA, MIOA
COICM	CMIA-O	MIIA (Maximum Influence In-Arborescence), MIOA(Maximum Influence Out-Arborescence)	MIIA, MIOA

der to efficiently compute the blocked negative influence of any node, we first compute negative activation probabilities of a node u in $MIIA(u, \theta, d)$ with all possible d . Let $t(u)$ be the number of lengths of paths from negative seeds to u in $MIIA(u, \theta)$, and $D = \{d_N^i(u) | i = 1, \dots, t(u)\}$ be the corresponding length set. Then negative activation probability of u in each $MIIA(u, \theta, d_N^i(u))$, $ap^N(u, S_N, MIIA(u, \theta, d_N^i(u)))$, is first computed by Algorithm 1. Let $d_{N, \max}(u, S_P)$ be the largest length in D satisfying the condition that it is smaller than $d_c(u, S_P)$. If no length in D is smaller than $d_c(u, S_P)$, $d_{N, \max}(u, S_P) = 0$ and $ap^N(u, S_N, MIIA(u, \theta, d_{N, \max}(u, S_P))) = 0$. Now the incremental blocked negative influence $Declnf(w, u, S_P)$ of w can be efficiently computed as

$$Declnf(w, u, S_P) = ap^N(u, S_N, MIIA(u, \theta, d_{N, \max}(u, S_P))) - ap^N(u, S_N, MIIA(u, \theta, d_{N, \max}(u, S_P \cup \{w\}))). \quad (1)$$

In Fig. 1, $t(u) = 2$ and $D = \{1, 3\}$. Assume that $d_c(u, S_P) = 2$, then $d_{N, \max}(u, S_P) = 1$. If $d_P(w, u) = 1$, then $d_{N, \max}(u, S_P \cup \{w\}) = 0$.

It is easy to infer that if $d_P(w, u) > d_{N, \max}(u, S_P)$, then $d_{N, \max}(u, S_P) = d_{N, \max}(u, S_P \cup \{w\})$, and $Declnf(w, u, S_P) = 0$ accordingly. Thus when computing the incremental blocked negative influence from the node u , we only consider set of nodes whose length of shortest path to u are no larger than $d_{N, \max}(u, S_P)$. Such set of nodes is denoted as Positive Influence In Set, $PIIS(u, d_{N, \max}(u, S_P))$, which can be constructed by the breadth first search (BFS) method from u . BFS method recursively searches neighbors of any searched node and adds them to $PIIS(u, d_{N, \max}(u, S_P))$ until the length of the shortest path from a searched node to u is larger than $d_{N, \max}(u, S_P)$.

Once we have the computation of blocked negative influence of any node as given in Eq. (1), we can design the algorithm CMIA-H for positive seed selection. The complete pseudocode description of CMIA-H is presented in Algorithm 2. Since the negative influence of each negative seed u is assumed to only propagate in $MIOA(u, \theta)$, the negative influence diffusion range $NegS$ of negative seed set S_N is first determined in lines 3–6. For each node u in the diffusion range, its negative activation probability in each $MIIA(u, \theta, d_N^i(u))$ is first computed by Algorithm 1. Then for each node in $PIIS(u, d_{N, \max}(u, S_P))$, increase its blocked negative influence with contribution from node u in lines 12–14. The node with maximum total blocked negative influence is iteratively selected as the positive seed in lines 16–32. After a new seed u is selected in line 17, the blocked negative influence of correlated nodes that may influence some nodes that are also influenced by u are updated in lines 19–31. The Positive Influence Out Set $PIOS(u)$ contains nodes to which u may have positive influence, i.e., $v \in PIOS(u)$ if and only if $u \in PIIS(v, d_{N, \max}(v, S_P))$. $PIOS(u)$ can be constructed by a simple BFS from u to contain all nodes v satisfying $d_P(u, v) \leq d_{N, \max}(v, S_P)$. All nodes $w \in PIIS(v, d_{N, \max}(v, S_P))$ with $v \in PIOS(u)$ need to update their $Declnf(w)$. The update is done by first subtracting old blocked negative influence of w computed before u is added into S_P (line 21), and adding new blocked negative influence of w computed after u is added into S_P (line 29). In order to avoid that the influence propagation in MIIA, MIOA, PIIS and PIOS is blocked by any seed, we adopt the idea from PMIA method [14] to construct them. Let $G(S)$ be the subgraph of G induced by $V \setminus S$. MIOA, PIIS and PIOS are all constructed by algorithms mentioned above on graph

Algorithm 2 CMIA-H(G, S_N, k, θ).

```

1:  $S_P = \emptyset$ ;  $NegS = \emptyset$ ;
2: Set  $Declnf(v) = 0$ ,  $d_c(v, S_P) = +\infty$  for each node  $v \in V$ ;
3: for each node  $u \in S_N$  do
4:   construct  $MIOA(u, \theta)$ ;
5:    $NegS = NegS \cup (MIOA(u, \theta) \setminus S_N)$ ;
6: end for
7: for each node  $u \in NegS$  do
8:   construct  $MIIA(u, \theta)$ ;
9:   compute  $D = \{d_N^i(u) | i = 1, \dots, t(u)\}$ ;
10:  compute  $ap^N(u, S_N, MIIA(u, \theta, d_N^i(u)))$ ,  $\forall d_N^i(u) \in D$ ;
11:  construct  $PIIS(u, d_{N, \max}(u, S_P))$ ;
12:  for each node  $v \in PIIS(u, d_{N, \max}(u, S_P))$  do
13:     $Declnf(v) += ap^N(u, S_N, MIIA(u, \theta, d_{N, \max}(u, S_P))) -$ 
       $ap^N(u, S_N, MIIA(u, \theta, d_{N, \max}(u, S_P \cup \{v\})))$ ;
14:  end for
15: end for
16: for  $i = 1$  to  $k$  do
17:    $u = \arg \max_{v \in V \setminus (S_P \cup S_N)} Declnf(v)$ ;
18:   construct  $PIOS(u)$ ;
19:   for each node  $v \in PIOS(u)$  do
20:     for each node  $w \in PIIS(v, d_{N, \max}(v, S_P))$  do
21:        $Declnf(w) -= ap^N(v, S_N, MIIA(v, \theta, d_{N, \max}(v, S_P))) -$ 
         $ap^N(v, S_N, MIIA(v, \theta, d_{N, \max}(v, S_P \cup \{w\})))$ ;
22:     end for
23:   end for
24:    $S_P = S_P \cup u$ ;
25:   for each node  $v \in PIOS(u) \setminus \{u\}$  do
26:     update  $d_c(v, S_P)$ ;
27:     construct  $PIIS(v, d_{N, \max}(v, S_P))$ ;
28:     for each node  $w \in PIIS(v, d_{N, \max}(v, S_P))$  do
29:        $Declnf(w) += ap^N(v, S_N, MIIA(v, \theta, d_{N, \max}(v, S_P))) -$ 
         $ap^N(v, S_N, MIIA(v, \theta, d_{N, \max}(v, S_P \cup \{w\})))$ ;
30:     end for
31:   end for
32: end for
33: return  $S_P$ ;

```

$G(S_P \cup S_N)$. $MIIA(u, \theta)$ is constructed by a Dijkstra algorithm started from u traversing inward edges. Whenever the Dijkstra algorithm hits a negative seed v , it stops this branch and does not go further on the in-neighbors of v .

The complexity of CMIA-H is given as Theorem 1.

Theorem 1. Let $n = |V|$, $n_N = |S_N|$, $n_{\theta}^N = \max_u |MIOA(u, \theta)|$, $n_{\theta}^N = \max_u |MIIA(u, \theta)|$, $n_i^P = \max_u |PIIS(u, d_{N, \max}(u, \emptyset))|$, and $n_o^P = \max_u |PIOS(u)|$. Let t_{θ}^N be the maximum time of construction of $MIIA(u, \theta)$ or $MIOA(u, \theta)$ for any u , t_i^P and t_o^P be the maximum time to construct $PIIS(u, d_{N, \max}(u, \emptyset))$ and $PIOS(u)$ for any u , respectively. The time complexity of CMIA-H is $O((n_N t_{\theta}^N)^2 + n_N t_{\theta}^N t_i^P + k t_o^P t_i^P \log n)$, and the space complexity of CMIA-H is $O(n_N n_{\theta}^N (n_N + n_i^P) + n)$.

Proof. Since the constructions of MIIA, MIOA, PIIS and PIOS have linear time complexity with their size, we have $n_{\theta}^N = O(t_{\theta}^N)$, $n_{\theta}^N = O(t_{\theta}^N)$, $n_i^P = O(t_i^P)$, and $n_o^P = O(t_o^P)$. Initially, lines 3–6 take

$O(n_N t_\theta^N)$ time to construct the negative influence diffusion range $NegS$. The size of $NegS$ is $O(n_N n_{\theta}^N)$. Since the maximum possible size of D is n_N , line 10 takes $O(n_N n_{\theta}^N)$ time to compute $ap^N(u, S_N, MIIA(u, \theta, d_N^i(u)))$ by Algorithm 1 for all $d_N^i(u) \in D$. We use a max-heap to store $Declnf(v)$, and its initialization time is $O(n)$. Overall, lines 7–15 take $O(n_N n_{\theta}^N (t_\theta^N + n_N n_{\theta}^N + t_i^P))$ which can be simplified as $O(n_N t_\theta^N (n_N t_\theta^N + t_i^P))$. In one iteration of the main loop, selecting the new seed from the max-heap takes constant time, and updating one $Declnf(v)$ on the max-heap takes $O(\log n)$ time. Overall, all iterations take $O(kn_o^P (n_i^P \log n + t_i^P) + kt_o^P)$ which can be simplified as $O(kt_o^P t_i^P \log n)$. Thus, the total time complexity of the algorithm is $O((n_N t_\theta^N)^2 + n_N t_\theta^N t_i^P + kt_o^P t_i^P \log n)$. It has the logarithm relation with n , i.e., the number of nodes of the whole network.

For space complexity, $NegS$, $ap^N(u, S_N, MIIA(u, \theta, d_N^i(u)))$ for every node $u \in NegS$ and every $d_N^i(u) \in D$, $PIIS(u, d_N, max(u, S_P))$ for every node $u \in NegS$, and a max-heap storing $Declnf(v)$ for any $v \in V$ are stored. The size of $NegS$ is $O(n_N n_{\theta}^N)$ and the maximum possible size of D is n_N , so the space complexity is $O(n_N n_{\theta}^N (n_N + n_i^P) + n)$. $\square$

4.2. CMIA-O algorithm for COICM

In COICM, positive propagation probability and negative propagation probability are equal for every edge. Every edge (u, v) is associated with one propagation probability $q_{u,v}$ to represent both positive and negative propagation probability. Thus positive and negative influence are assumed to propagate in the same MIIA and MIOA in COICM. Different from MCICM with high-effectiveness property where positive seeds either totally block or have no impact on the negative influence from negative seeds to other nodes, the positive seeds block the negative influence with some probability in COICM. Thus, its more complicated to compute the negative activation probability of any node in MIIA in COICM. Inspired by [1] which addresses the IBM in CLTM, we propose a dynamic programming method to compute the negative activation probability of the node u in $MIIA(u, \theta)$ under negative seed set S_N and positive seed set S_P , denoted as $ap^N(u, S_N, S_P, MIIA(u, \theta))$. However, the computation procedure is different from that in [1].

Given the negative seed set S_N and the positive seed set S_P , $ap^N(u, S_N, S_P, MIIA(u, \theta))$ need to be computed first. Let $p^N(u, t)$ and $p^P(u, t)$ be the probabilities that u becomes negatively activated and positively activated exactly at time t in $MIIA(u, \theta)$, respectively. Let $ap^N(u, t)$ and $ap^P(u, t)$ be the probabilities that u is negatively active and positively active after time t in $MIIA(u, \theta)$, respectively. Let $N^{in}(v)$ denote the set of in-neighbors of v in $MIIA(u, \theta)$. Assume that the negative propagation in $MIIA(u, \theta)$ stops at time T , then $ap^N(u, S_N, S_P, MIIA(u, \theta)) = ap^N(u, T)$. According to the definition of COICM, we have following relationships between probabilities defined above for any $v \in MIIA(u, \theta) \setminus (S_N \cup S_P)$ and any $t \in [1, T]$:

$$p^P(v, t) = \left(1 - \prod_{w \in N^{in}(v)} (1 - p^P(w, t-1)q_{w,v}) \right) (1 - ap^N(v, t-1))(1 - ap^P(v, t-1)); \quad (2)$$

$$p^N(v, t) = \prod_{w \in N^{in}(v)} (1 - p^P(w, t-1)q_{w,v}) \times \left(1 - \prod_{w \in N^{in}(v)} (1 - p^N(w, t-1)q_{w,v}) \right) (1 - ap^N(v, t-1))(1 - ap^P(v, t-1)); \quad (3)$$

$$ap^P(v, t) = ap^P(v, t-1) + p^P(v, t); \quad (4)$$

$$ap^N(v, t) = ap^N(v, t-1) + p^N(v, t); \quad (5)$$

Eq. (2) comes from the fact that node v becomes positively activated at time t if and only if at least one neighbor of v becomes positively activated at time $t-1$ and successfully positively activates v , while v is neither negatively active nor positively active after time $t-1$. The case for Eq. (3) is similar except that negative activation requires one more condition that no neighbor of v can successfully positively activate v at time t . Eqs. (4) and (5) are directly derived from their meaning that the activation probability after time t equals to the activation probability after time $t-1$ plus the activation probability at time t .

The boundary conditions of the above equations are (a) for any $v \in S_P$, $p^P(v, 0) = 1$, $p^P(v, t) = 0$ for all $t \geq 1$, $ap^P(v, t) = 1$ for all $t \geq 0$, $p^N(v, t) = ap^N(v, t) = 0$ for all $t \geq 0$; (b) for any $v \in S_N$, $p^N(v, 0) = 1$, $p^N(v, t) = 0$ for all $t \geq 1$, $ap^N(v, t) = 1$ for all $t \geq 0$, $p^P(v, t) = ap^P(v, t) = 0$ for all $t \geq 0$; (c) for any $v \notin S_P \cup S_N$, $p^P(v, 0) = ap^P(v, 0) = p^N(v, 0) = ap^N(v, 0) = 0$. Based on Eqs. (2)–(5) and the corresponding boundary conditions, $ap^N(u, S_N, S_P, MIIA(u, \theta))$ can be computed by a dynamic programming algorithm described in Algorithm 3.

Algorithm 3 $ap^N(u, S_N, S_P, MIIA(u, \theta))$.

```

1:  $Z_0^P = S_P \cap MIIA(u, \theta)$ ;  $Z_0^N = S_N \cap MIIA(u, \theta)$ ;  $Z_t^P = \emptyset$  for  $t \geq 1$ ;  $Z_t^N = \emptyset$  for  $t \geq 1$ ;
2: initialize  $p^P(v, t)$ ,  $p^N(v, t)$ ,  $ap^P(v, t)$ ,  $ap^N(v, t)$  for all  $v$  and  $t$  according to the boundary condition.
3:  $t = 0$ ;
4: while  $Z_t^N \neq \emptyset$  do
5:   set  $tempP(v) = tempN(v) = 1$  for all  $v \in MIIA(u, \theta)$ ;
6:   for  $v \in Z_t^P$  do
7:      $w = parent(v)$ ;
8:      $Z_{t+1}^P = Z_{t+1}^P \cup \{w\}$ ;
9:      $tempP(w) \times = (1 - p^P(v, t)q_{v,w})$ ;
10:  end for
11:  for  $v \in Z_{t+1}^P$  do
12:     $p^P(v, t+1) = (1 - tempP(v))(1 - ap^N(v, t))(1 - ap^P(v, t))$ ;
13:     $ap^P(v, \tau) = ap^P(v, t) + p^P(v, t+1)$  for  $\tau > t$ ;
14:  end for
15:  for  $v \in Z_t^N$  do
16:     $w = parent(v)$ ;
17:     $Z_{t+1}^N = Z_{t+1}^N \cup \{w\}$ ;
18:     $tempN(w) \times = (1 - p^N(v, t)q_{v,w})$ ;
19:  end for
20:  for  $v \in Z_{t+1}^N$  do
21:     $p^N(v, t+1) = tempP(v)(1 - tempN(v))(1 - ap^N(v, t))(1 - ap^P(v, t))$ ;
22:     $ap^N(v, \tau) = ap^N(v, t) + p^N(v, t+1)$  for  $\tau > t$ ;
23:  end for
24:   $t = t + 1$ ;
25: end while
26: return  $ap^N(u, t)$ ;

```

The steps in main loop of the Algorithm 3 simulate the influence propagation in $MIIA(u, \theta)$. Since positive influence has priority over negative influence, we assume positive influence diffuse first at each time. $parent(v)$ in lines 7 and 16 denotes the out neighbor of v in $MIIA(u, \theta)$. Every node in $MIIA(u, \theta)$ except u has exactly one out neighbor in $MIIA(u, \theta)$. $tempP(v)$ records the probability that v has not been positively activated at current iteration, and similar for $tempN(v)$. They are used to compute $p^P(v, t+1)$ in line 12 and $p^N(v, t+1)$ in line 21. $ap^P(v, t+1)$ are computed directly

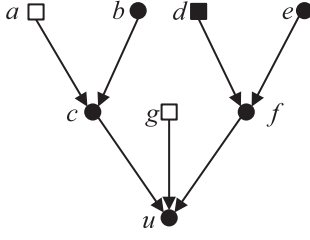


Fig. 2. A toy example of $MIIA(u, \theta)$. a, g are negative seeds, and d is a positive seed.

by Eqs. (4) in line 13, and similar for $ap^N(v, t+1)$ in line 22. Both positive propagation and negative propagation visit each edge in $MIIA(u, \theta)$ at most one time. Once an edge is visited by a propagation, it takes constant time to compute the activation probability of the node pointed to by such edge. Thus, the Algorithm 3 has the time complexity $O(n_{i\theta})$, where $n_{i\theta} = \max_u |MIIA(u, \theta)|$.

As an example, we show the computation of $ap^N(u, S_N, S_p, MIIA(u, \theta))$ in $MIIA(u, \theta)$ in Fig. 2. In the example, d is the positive seed and a and g are two negative seeds. In initialization, $p^P(d, 0) = 1$, $ap^P(d, t) = 1$ for all $t \geq 0$, $p^N(a, 0) = 1$, $ap^N(a, t) = 1$ for all $t \geq 0$, $p^N(g, 0) = 1$, $ap^N(g, t) = 1$ for all $t \geq 0$, and all other values are set to 0. In the first iteration, we start from the positive seed d to compute $p^P(f, 1)$ and $ap^P(f, \tau)$ for $\tau > 0$. Then we start from the negative seeds a and g to compute $p^N(c, 1)$, $ap^N(c, \tau)$ for $\tau > 0$, $p^N(u, 1)$ and $ap^N(u, \tau)$ for $\tau > 0$. In the second iteration, we start from f to compute $p^P(u, 2)$ and $ap^P(u, \tau)$ for $\tau > 1$, which use the computed $ap^N(u, 1)$. Then we start from c to compute $p^N(u, 2)$ and $ap^N(u, \tau)$ for $\tau > 1$, which use the computed $ap^N(u, 1)$. Since u has no out neighbor, the computation finishes. With the negative activation probability of a node u , the incremental blocked negative influence of a node w from u after adding w to the positive seed set S_p , $Declnf(w, u, S_p)$, can be computed as

$$Declnf(w, u, S_p) = ap^N(u, S_N, S_p, MIIA(u, \theta)) - ap^N(u, S_N, S_p \cup \{w\}, MIIA(u, \theta)).$$

Once we have the computation of blocked negative influence of any node, we can design the algorithm CMIA-O for positive seed selection. The complete pseudocode description of CMIA-O is presented in Algorithm 4. The negative influence diffusion range $NegS$ of negative seed set S_N is first determined in lines 3–6. For each node u in the diffusion range, we construct its $MIIA(u, \theta)$ and compute its $ap^N(u, S_N, S_p, MIIA(u, \theta))$ by Algorithm 3. For each node $v \in MIIA(u, \theta)$, we compute $ap^N(u, S_N, S_p \cup \{v\}, MIIA(u, \theta))$ when adding v as a positive seed, and increase blocked negative influence of v with contribution from node u in line 12. The node with maximum blocked negative influence is iteratively selected as the positive seed in lines 15–32. After a new seed u is selected in line 16, the blocked negative influence of correlated nodes that may influence some nodes that are also influenced by u are updated in lines 17–31. $MIOA(u, \theta)$ contains nodes to which u may have positive influence. All nodes $w \in MIIA(v, \theta)$ with $v \in MIOA(u, \theta)$ need to update their $Declnf(w)$. The update is done by first subtracting old blocked negative influence of w from v computed before u is added into S_p (line 20), and adding new blocked negative influence of w from v computed after u is added into S_p (line 29). We also adopt the idea from PMIA method [14] to construct MIOA and MIIA. $MIOA(u, \theta)$ is constructed on graph $G(S_p \cup S_N)$. $MIIA(u, \theta)$ is constructed by a Dijkstra algorithm started from u traversing inward edges. Whenever the Dijkstra algorithm hits a negative or positive seed node v , it stops this branch and does not go further on the in-neighbors of v . After the Dijkstra algorithm completes, all ineffective positive seeds are removed from the computed in-arborescence.

The complexity of CMIA-O is given as Theorem 2.

Algorithm 4 CMIA-O(G, S_N, k, θ).

```

1:  $S_p = \emptyset$ ;  $NegS = \emptyset$ ;
2: Set  $Declnf(v) = 0$  for each node  $v \in V$ ;
3: for each node  $u \in S_N$  do
4:   construct  $MIOA(u, \theta)$ ;
5:    $NegS = NegS \cup (MIOA(u, \theta) \setminus S_N)$ ;
6: end for
7: for each node  $u \in NegS$  do
8:   construct  $MIIA(u, \theta)$ ;
9:   compute  $ap^N(u, S_N, S_p, MIIA(u, \theta))$ ;
10:  for each node  $v \in MIIA(u, \theta)$  do
11:    compute  $ap^N(u, S_N, S_p \cup \{v\}, MIIA(u, \theta))$ ;
12:     $Declnf(v) += ap^N(u, S_N, S_p, MIIA(u, \theta)) - ap^N(u, S_N, S_p \cup \{v\}, MIIA(u, \theta))$ ;
13:  end for
14: end for
15: for  $i = 1$  to  $k$  do
16:    $u = \arg \max_{v \in V \setminus (S_p \cup S_N)} Declnf(v)$ ;
17:   construct  $MIOA(u, \theta)$ ;
18:   for each node  $v \in MIOA(u, \theta)$  do
19:     for each node  $w \in MIIA(v, \theta)$  do
20:        $Declnf(w) -= ap^N(v, S_N, S_p, MIIA(v, \theta)) - ap^N(v, S_N, S_p \cup \{w\}, MIIA(v, \theta))$ ;
21:     end for
22:   end for
23:    $S_p = S_p \cup u$ ;
24:   for each node  $v \in MIOA(u, \theta) \setminus \{u\}$  do
25:     construct  $MIIA(v, \theta)$ ;
26:     compute  $ap^N(v, S_N, S_p, MIIA(v, \theta))$ ;
27:     for each node  $w \in MIIA(v, \theta)$  do
28:       compute  $ap^N(v, S_N, S_p \cup \{w\}, MIIA(v, \theta))$ ;
29:        $Declnf(w) += ap^N(v, S_N, S_p, MIIA(v, \theta)) - ap^N(v, S_N, S_p \cup \{w\}, MIIA(v, \theta))$ ;
30:     end for
31:   end for
32: end for
33: return  $S_p$ ;

```

Theorem 2. Let $n = |V|$, $n_N = |S_N|$, $n_{o\theta} = \max_u |MIOA(u, \theta)|$, $n_{i\theta} = \max_u |MIIA(u, \theta)|$. Let t_θ be the maximum time of construction of $MIIA(u, \theta)$ or $MIOA(u, \theta)$ for any u . The time complexity of CMIA-O is $O(n_N t_\theta^3 + k t_\theta^2 (t_\theta + \log n))$, and the space complexity of CMIA-O is $O(n_N n_{o\theta} n_{i\theta} + n)$.

Proof. Since the constructions of MIIA and MIOA have linear time complexity with their size, we have $n_{o\theta} = O(t_\theta)$, $n_{i\theta} = O(t_\theta)$. Initially, lines 3–6 take $O(n_N t_\theta)$ time to construct negative influence diffusion range $NegS$. The size of $NegS$ is $O(n_N n_{o\theta})$. The computation of $ap^N(u, S_N, S_p, MIIA(u, \theta))$ by Algorithm 3 takes $O(n_{i\theta})$ time. A max-heap is used to store $Declnf(v)$, and its initialization time is $O(n)$. Overall, lines 7–14 take $O(n_N n_{o\theta} (t_\theta + n_{i\theta}^2))$ which can be simplified as $O(n_N t_\theta^3)$. In one iteration of the main loop, selecting the new seed from the max-heap takes constant time, and updating one $Declnf(v)$ on the max-heap takes $O(\log n)$ time. Overall, all iterations take $O(k n_{o\theta} (n_{i\theta} (n_{i\theta} + \log n) + t_\theta))$ which can be simplified as $O(k t_\theta^2 (t_\theta + \log n))$. Thus, the total time complexity of the algorithm is $O(n_N t_\theta^3 + k t_\theta^2 (t_\theta + \log n))$. It has the logarithm relation with n , i.e., the number of nodes of the whole network.

For space complexity, $NegS$, $ap^N(u, S_N, S_p \cup \{v\}, MIIA(u, \theta))$ for every $u \in NegS$ and every $v \in MIIA(u, \theta)$, $MIIA(u, \theta)$ for every $u \in NegS$, and a max-heap storing $Declnf(v)$ are stored. The size of $NegS$ is $O(n_N n_{o\theta})$, so the space complexity is $O(n_N n_{o\theta} n_{i\theta} + n)$. $\square$

Table 3
Statistics of the four real-world networks.

Dataset	EMAIL	NetHEPT	NetPHY	DBLP
#Node	1.13K	15.2K	37.2K	655K
#Edge	5.45K	58.9K	232K	1.99M
Average Degree	9.62	7.75	12.5	6.08

5. Experiments

In this section, we conduct experiments on four real-world networks and a set of synthetic networks to evaluate the efficiency and effectiveness of our algorithms for IBM problem under the MCICM with high-effectiveness property and COICM. All experiments are carried out on a computer running 64bit Windows 10 and equipped with 3.4GHz Quad-Core Intel i7-6700 and 16GB RAM. All experiment code is written in C++.

5.1. Experiment setting

Four real-world networks are used. The EMAIL network of University Rovira i Virgili [29] is built by regarding each email address as a node and linking two nodes if there is an email communication between them. The NetHEPT and NetPHY are both academic collaboration networks extracted from “High Energy Physics Theory” section and “Physics” section of the e-print arXiv (<http://www.arXiv.org>), respectively. They are the same datasets used in [30]. The DBLP is a much larger collaboration network extracted from Computer Science Bibliography Database (<http://dblp.uni-trier.de>). The nodes in all collaboration networks are authors and an edge between two nodes means the two authors coauthored at least one paper. Some basic statistics of these networks are shown in Table 3. Finally, to evaluate the scalability of the algorithms, we adopt a set of LFR benchmarks [31] of different sizes which have several similar features to social networks. The default parameters of LFR are set as follows, degree distribution exponent $\tau_1 = 2$, community size distribution exponent $\tau_2 = 1$, average node degree $d_{avg} = 8$, maximum node degree $d_{max} = 20$, minimum community size $c_{min} = 10$, maximum community size $c_{max} = 25$ and mixing parameter $\mu = 0.3$.

In MCICM with high-effectiveness property, negative propagation probability need to be generated, while positive propagation probability is set as 1. In COICM, negative propagation probability is equal to the positive one. Thus, in both models, only one type of propagation probability need to be generated for every edge. We use the following two models which is the same as used in [14] to generate propagation probabilities for both MCICM with high-effectiveness property and COICM.

TRIVALENCY model: we uniformly select a probability from the set $\{0.2, 0.05, 0.01\}$ for every edge, which corresponds to high, medium, and low influences.

WC model: the weighted cascade model proposed in [13] where $q_{u,v}$ ($q_{N,u,v}$) for an edge (u,v) is $1/d_v$, where d_v is the in-degree of v .

Our algorithms are compared with both greedy algorithms and several heuristics. The following is a list of evaluated algorithms.

CMIA-H and CMIA-O: our algorithms for two models with $\theta = 1/100$. Experiments show that $\theta < 1/100$ will not have significant improvement for influence blocking effect on all evaluated datasets;

Greedy-H and Greedy-O: greedy algorithms under two propagation models, respectively, with lazy-forward optimization of [15], and 10,000 Monte-Carlo simulation runs for each influence estimate;

Proximity: a heuristic proposed in [1] where the direct out-neighbors of negative seeds are selected as positive seeds. All di-

rect out-neighbors are sorted by the negative activation probability from their direct in-neighbor negative seeds, and top k nodes with largest negative activation probabilities are selected as the positive seeds.

Degree: top k nodes with largest degrees are selected as positive seeds.

Random: selecting nodes at random as positive seeds.

For each negative seed set and positive seed set, we run the propagation simulation 10,000 times and take their average as the expected negative influence spread. As suggested in [14], percentage differences on blocked negative influence between different methods reported below are the average of percentage differences from selecting one positive seed to selecting maximum number of positive seeds.

5.2. Results with the greedy algorithm

Since the greedy algorithms are only feasible to run on small networks, the experiments are first run on EMAIL network to include the comparison with greedy algorithms. We randomly choose 50 nodes as negative seeds and aim to select 200 positive seeds. The results are illustrated in Fig. 3. Fig. 3(a) and (c) show the results under the MCICM with high-effectiveness property. All of the methods can protect a large portion of the population with a few positive seeds under the MCICM with high-effectiveness property. In Fig. 3(a), CMIA-H matches the negative influence blocking performance of Greedy-H for the TRIVALENCY model and it is 4.3%, 1.3% and 1.8% better than Random, Degree and Proximity, respectively. In Fig. 3(c), CMIA-H has the best performance for the WC model and it is 16.8%, 8.4%, 5.2% and 5.7% better than Random, Degree, Proximity and Greedy-H, respectively. Fig. 3(b) and (d) show the results under the COICM. In Fig. 3(b), CMIA-O has the best performance for the TRIVALENCY model and it is 32.8%, 4.8%, 8.4% and 3.0% better than Random, Degree, Proximity and Greedy-O, respectively. In Fig. 3(d), CMIA-O has the best performance for the WC model, and it is 57.6%, 7.5%, 4.0% and 6.3% better than Random, Degree, Proximity and Greedy-O, respectively. Fig. 3(e) shows the running time of each method on EMAIL network. Both Greedy-H and Greedy-O take more than 6 hours, while CMIA-O takes several seconds and CMIA-H even takes less than one second. CMIA is much faster than the Greedy algorithm, with more than three orders of magnitude speedup.

The scalability of different methods is tested on a family of LFR networks [31] of increasing sizes. The networks are generated with doubling number of nodes, from 0.2K up to 25.6K. Each size has 10 different networks and the running time is the average of the runs on 10 networks. The WC model is used for scalability test. We randomly select 50 nodes as negative seeds and aim to find 50 positive seeds. The scalability test results are illustrated in Fig. 4. Algorithms Greedy-H and Greedy-O are too slow to run on networks whose sizes are larger than 1600. The results show that CMIA is more than four orders of magnitude faster than the greedy algorithms. Moreover, CMIA has sublinear relationship with the size of the network. This is because we only consider nodes in the negative influence diffusion range $NegS$ rather than in the whole network when computing the negative activation probability. Overall, CMIA-H and CMIA-O have matched influence blocking performance with the greedy algorithms, while are more than several orders of magnitude faster to run than the greedy algorithms.

5.3. Results on larger dataset without the greedy algorithm

The experiments are also conducted on three larger datasets. Since the greedy algorithms cannot return results on three larger datasets within 100 hours, they are not included in the comparisons in this subsection. In each case, 200 nodes are randomly

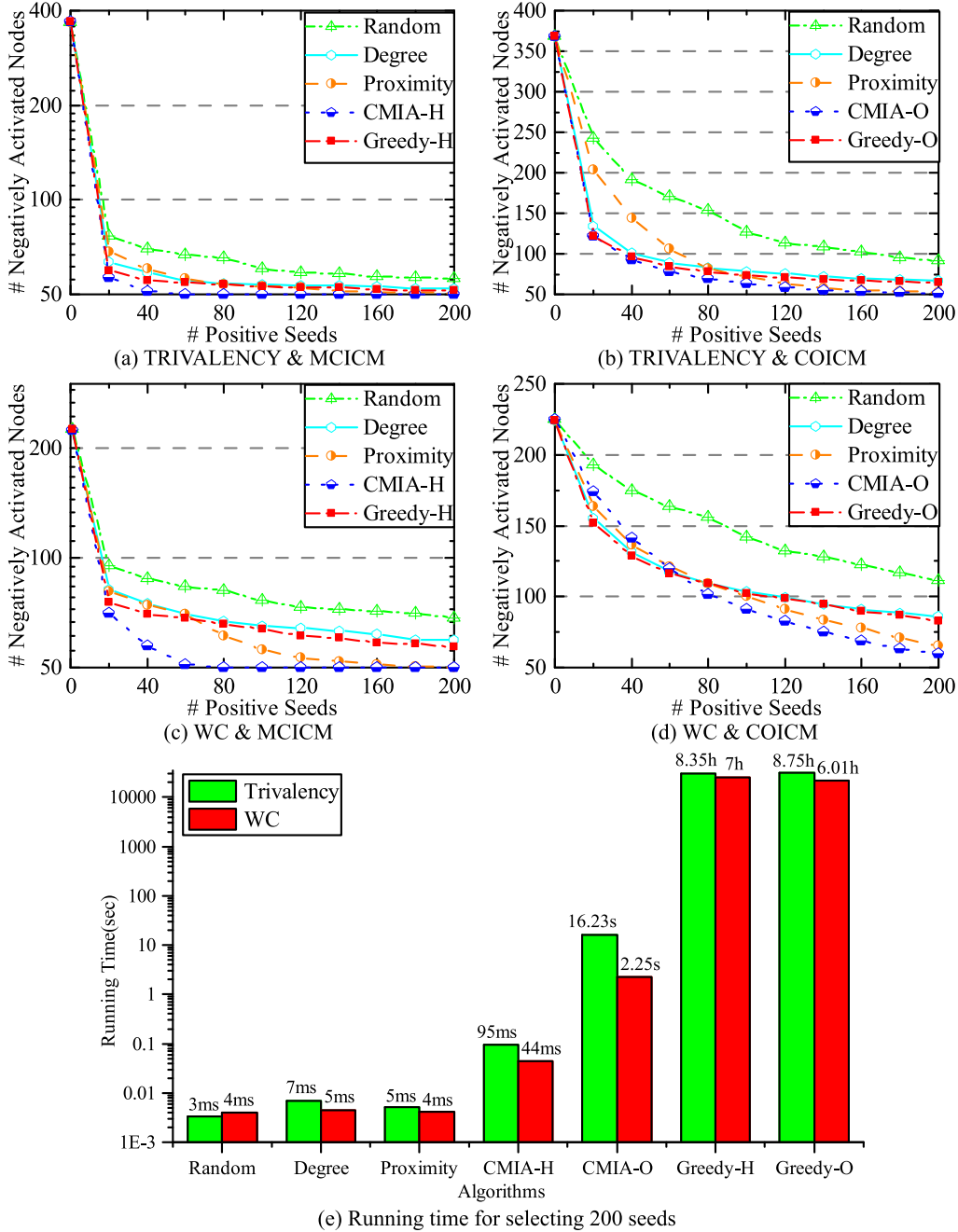


Fig. 3. Results with the greedy algorithm on EMAIL network.

selected as negative seeds and we aim to select 200 positive seeds. The results for TRIVALENCY model are illustrated in Fig. 5. Fig. 5(a), (c) and (e) show the results under the MCICM with high-effectiveness property on three larger networks, respectively. All of the methods can protect a large portion of the population with a few positive seeds. CMIA-H is on average 9.1%, 3.8% and 2.8% better than Random, Degree and Proximity, respectively. Fig. 5(b), (d) and (f) show the results under the COICM on three larger networks, respectively. CMIA-O is on average 100%, 86.7% and 17.7% better than Random, Degree and Proximity, respectively. The results for WC model are illustrated in Fig. 6. Fig. 6(a), (c) and (e) show the results under the MCICM with high-effectiveness property on three larger networks, respectively. CMIA-H is on average 89.5%, 51.9% and 55.5% better than Random, Degree and Proximity, respectively. Fig. 6(b), (d) and (f) show the results under the COICM on three

larger networks, respectively. CMIA-O is on average 1000%, 86.7% and 17.7% better than Random, Degree and Proximity, respectively. Fig. 7 shows the CMIA-H and CMIA-O are efficient to run on all three larger real-world networks. In a word, CMIA-H and CMIA-O can scale to very large networks, and they consistently outperform other scalable heuristics in negative influence blocking performance with a significant margin.

6. Conclusion

The IBM problem aims to find a set of positive seeds initiating positive propagation to maximize the blocked negative influence of a given negative seed set. Though greedy algorithms for IBM problem under both MCICM with high-effectiveness property and COICM have approximation guarantee, they are too slow to run

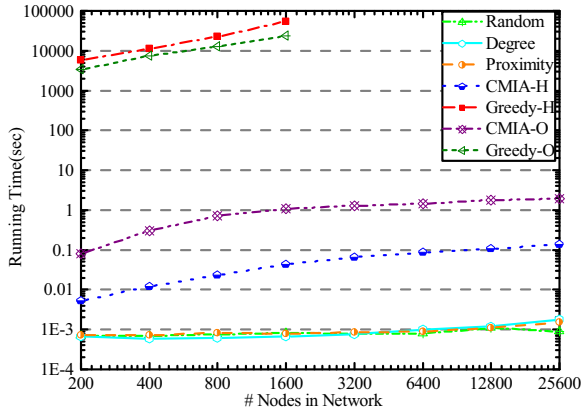


Fig. 4. Scalability of different methods in synthetic datasets. Each data point is an average of ten runs.

on large networks. In this paper, we address the scalability issue of solving the IBM problem under MCICM with high-effectiveness property and COICM, respectively. Two efficient heuristics CMIA-H and CMIA-O are proposed by designing two efficient blocked negative influence computation procedures based on MIA structure,

respectively. The computation procedures in CMIA-H adopts the high-effectiveness property of the positive propagation to easily decide whether the influence from negative seeds to other nodes is totally blocked or not blocked at all and then compute the blocked negative influence. The computation procedures in CMIA-O adopts a dynamic programming method to efficiently compute the negative activation probability of any node and then compute the blocked negative influence. Once computing the blocked negative influence of any node, both CMIA-H and CMIA-O iteratively select the node with maximum blocked negative influence as the positive seed. Experiment results demonstrate that CMIA-H and CMIA-O have matched influence blocking performance to greedy algorithms, while are several orders of magnitude faster to run. On the other hand, they consistently outperform other scalable heuristics in negative influence blocking performance with a significant margin.

Though the proposed algorithms have matched influence blocking performance to greedy algorithms in the experiments, they lack approximation guarantee. One possible research direction is designing efficient algorithms for IBM problem that not only scale to large networks, but also have approximation guarantee. Moreover, studying IBM problem under other more realistic competitive propagation model is another challenging future work.

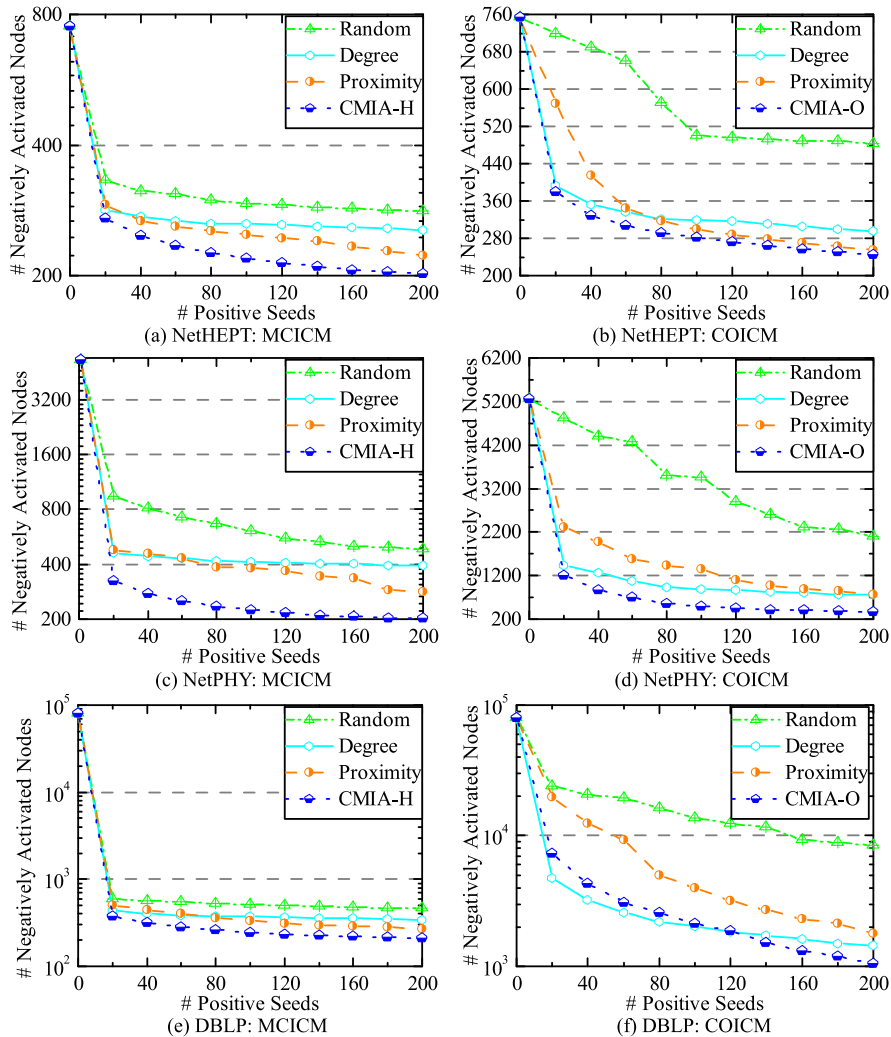


Fig. 5. Experiment results for TRIVALENCY model on three larger real-world networks.

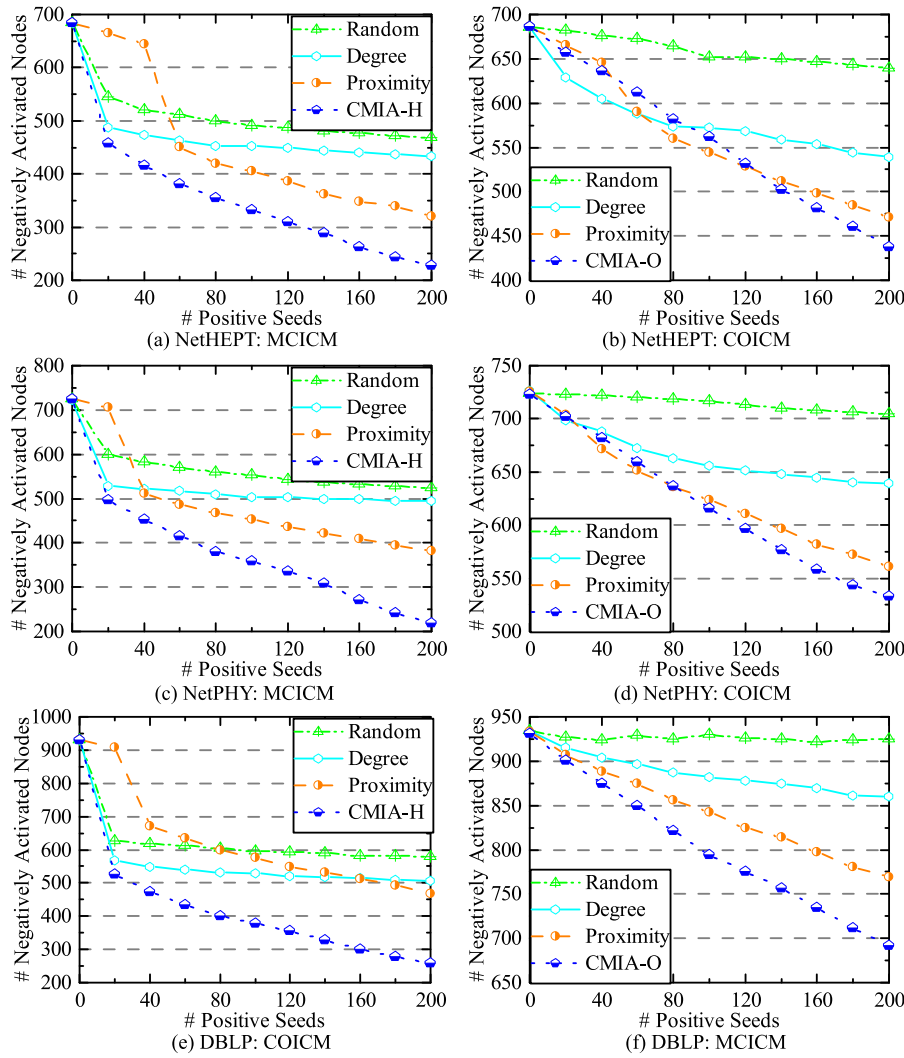


Fig. 6. Experiment results for WC model on three larger real-world networks.

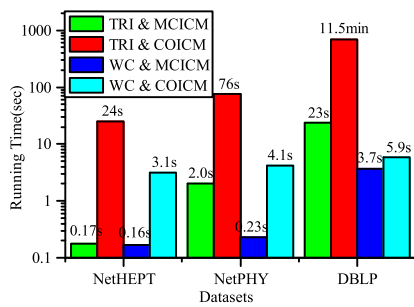


Fig. 7. Running time of CMIA-H and CMIA-O on three larger real-world networks.

Acknowledgments

This work is supported by National Key Basic Research Program of China (2013CB329603), National Natural Science Foundation of China (U1636105).

References

- [1] X. He, G. Song, W. Chen, Q. Jiang, Influence blocking maximization in social networks under the competitive linear threshold model, in: SDM, SIAM, 2012, pp. 463–474, doi:10.1137/1.9781611972825.40.
- [2] C. Budak, D. Agrawal, A. El Abbadi, Limiting the spread of misinformation in social networks, in: Proceedings of the 20th international conference on World wide web, ACM, 2011, pp. 665–674, doi:10.1145/1963405.1963499.
- [3] S. Wen, J. Jiang, Y. Xiang, S. Yu, W. Zhou, W. Jia, To shut them up or to clarify: restraining the spread of rumors in online social networks, IEEE Trans. Parallel Distrib. Syst. 25 (12) (2014) 3306–3316, doi:10.1109/TPDS.2013.2297115.
- [4] Z. He, Z. Cai, X. Wang, Modeling propagation dynamics and developing optimized countermeasures for rumor spreading in online social networks, in: Distributed Computing Systems (ICDCS), 2015 IEEE 35th International Conference on, IEEE, 2015, pp. 205–214, doi:10.1109/ICDCS.2015.29.
- [5] Y. Song, T.N. Dinh, Optimal containment of misinformation in social media: A scenario-based approach, in: International Conference on Combinatorial Optimization and Applications, Springer, 2014, pp. 547–556, doi:10.1007/978-3-319-12691-3_40.
- [6] P. Foster, 'bogus' AP tweet about explosion at the white house wipes billions off US markets, 2013, (<http://www.telegraph.co.uk/finance/markets/10013768/Bogus-AP-tweet-about-explosion-at-the-White-House-wipes-billions-off-US-markets.html>).
- [7] N.P. Nguyen, G. Yan, M.T. Thai, S. Eidenbenz, Containment of misinformation spread in online social networks, in: Proceedings of the 4th Annual ACM Web Science Conference, ACM, 2012, pp. 213–222, doi:10.1145/2380718.2380746.
- [8] L. Fan, Z. Lu, W. Wu, B. Thuraisingham, H. Ma, Y. Bi, Least cost rumor blocking in social networks, in: Distributed Computing Systems (ICDCS), 2013 IEEE 33rd International Conference on, IEEE, 2013, pp. 540–549, doi:10.1109/ICDCS.2013.34.
- [9] L. Fan, W. Wu, X. Zhai, K. Xing, W. Lee, D.-Z. Du, Maximizing rumor containment in social networks with constrained time, Social Netw. Anal. Min. 4 (1) (2014) 1–10, doi:10.1007/s13278-014-0214-4.
- [10] Y. Ping, Z. Cao, H. Zhu, Sybil-aware least cost rumor blocking in social networks, in: 2014 IEEE Global Communications Conference, IEEE, 2014, pp. 692–697, doi:10.1109/GLOCOM.2014.7036888.

- [11] N. Wang, L. Yu, N. Ding, D. Yang, Containment of misinformation propagation in online social networks with given deadline., in: PACIS, 2014, p. 46.
- [12] H. Zhang, H. Zhang, X. Li, M.T. Thai, Limiting the spread of misinformation while effectively raising awareness in social networks, in: International Conference on Computational Social Networks, Springer, 2015, pp. 35–47, doi:[10.1007/978-3-319-21786-4_4](https://doi.org/10.1007/978-3-319-21786-4_4).
- [13] D. Kempe, J. Kleinberg, E. Tardos, Maximizing the spread of influence through a social network, in: Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2003, pp. 137–146, doi:[10.1145/956750.956769](https://doi.org/10.1145/956750.956769).
- [14] W. Chen, C. Wang, Y. Wang, Scalable influence maximization for prevalent viral marketing in large-scale social networks, in: Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2010, pp. 1029–1038, doi:[10.1145/1835804.1835934](https://doi.org/10.1145/1835804.1835934).
- [15] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, N. Glance, Cost-effective outbreak detection in networks, in: Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, 2007, pp. 420–429, doi:[10.1145/1281192.1281239](https://doi.org/10.1145/1281192.1281239).
- [16] W. Chen, Y. Yuan, L. Zhang, Scalable influence maximization in social networks under the linear threshold model, in: 2010 IEEE International Conference on Data Mining, IEEE, 2010, pp. 88–97.
- [17] C. Borgs, M. Brautbar, J. Chayes, B. Lucier, Maximizing social influence in nearly optimal time, in: Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, Society for Industrial and Applied Mathematics, 2014, pp. 946–957.
- [18] Y. Tang, Y. Shi, X. Xiao, Influence maximization in near-linear time: amartingale approach, in: Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, ACM, 2015, pp. 1539–1554, doi:[10.1145/2723372.2723734](https://doi.org/10.1145/2723372.2723734).
- [19] C. Zhou, P. Zhang, W. Zang, L. Guo, On the upper bounds of spread for greedy algorithms in social network influence maximization, IEEE Trans. Knowl. Data Eng. 27 (10) (2015) 2770–2783, doi:[10.1109/TKDE.2015.2419659](https://doi.org/10.1109/TKDE.2015.2419659).
- [20] M. Kimura, K. Saito, H. Motoda, Blocking links to minimize contamination spread in a social network, ACM Trans. Knowl. Discovery Data (TKDD) 3 (2) (2009) 9, doi:[10.1145/1514888.1514892](https://doi.org/10.1145/1514888.1514892).
- [21] C.J. Kuhlman, G. Tuli, S. Swarup, M.V. Marathe, S. Ravi, Blocking simple and complex contagion by edge removal, in: 2013 IEEE 13th International Conference on Data Mining, IEEE, 2013, pp. 399–408, doi:[10.1109/ICDM.2013.47](https://doi.org/10.1109/ICDM.2013.47).
- [22] E.B. Khalil, B. Dilkina, L. Song, Scalable diffusion-aware optimization of network topology, in: Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, 2014, pp. 1226–1235, doi:[10.1145/2623330.2623704](https://doi.org/10.1145/2623330.2623704).
- [23] S. Saha, A. Adiga, B.A. Prakash, A.K.S. Vullikanti, Approximation algorithms for reducing the spectral radius to control epidemic spread, in: Proceedings of the 2015 SIAM International Conference on Data Mining, SIAM, 2015, pp. 568–576, doi:[10.1137/1.9781611974010.64](https://doi.org/10.1137/1.9781611974010.64).
- [24] Y. Zhang, A. Adiga, A. Vullikanti, B.A. Prakash, Controlling propagation at group scale on networks, in: Data Mining (ICDM), 2015 IEEE International Conference on, IEEE, 2015, pp. 619–628, doi:[10.1109/ICDM.2015.59](https://doi.org/10.1109/ICDM.2015.59).
- [25] M. Caleffi, I.F. Akyildiz, L. Paura, On the solution of the steiner tree NP-hard problem via physarum bionetwork, IEEE/ACM Trans. Netw. 23 (4) (2015) 1092–1106, doi:[10.1109/TNET.2014.2317911](https://doi.org/10.1109/TNET.2014.2317911).
- [26] A. Nazi, M. Raj, M. Di Francesco, P. Ghosh, S.K. Das, Efficient communications in wireless sensor networks based on biological robustness, in: Distributed Computing in Sensor Systems (DCOSS), 2016 International Conference on, IEEE, 2016, pp. 161–168, doi:[10.1109/DCOSS.2016.14](https://doi.org/10.1109/DCOSS.2016.14).
- [27] G. Cornuejols, M.L. Fisher, G.L. Nemhauser, Exceptional paperlocation of bank accounts to optimize float: an analytic study of exact and approximate algorithms, Manage. Sci. 23 (8) (1977) 789–810, doi:[10.1287/mnsc.23.8.789](https://doi.org/10.1287/mnsc.23.8.789).
- [28] G.L. Nemhauser, L.A. Wolsey, M.L. Fisher, An analysis of approximations for maximizing submodular set functions, Math. Program. 14 (1) (1978) 265–294, doi:[10.1007/BF01588971](https://doi.org/10.1007/BF01588971).
- [29] R. Guimera, L. Danon, A. Diaz-Guilera, F. Giralt, A. Arenas, Self-similar community structure in a network of human interactions, Phys. Rev. E 68 (6) (2003) 065103, doi:[10.1103/PhysRevE.68.065103](https://doi.org/10.1103/PhysRevE.68.065103).
- [30] W. Chen, Y. Wang, S. Yang, Efficient influence maximization in social networks, in: Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining, ACM, 2009, pp. 199–208, doi:[10.1145/1557019.1557047](https://doi.org/10.1145/1557019.1557047).
- [31] A. Lancichinetti, S. Fortunato, Benchmarks for testing community detection algorithms on directed and weighted graphs with overlapping communities, Phys. Rev. E 80 (1) (2009) 016118, doi:[10.1103/PhysRevE.80.016118](https://doi.org/10.1103/PhysRevE.80.016118).



Peng Wu is currently a PhD Student in Department of Electronic Engineering in Shanghai Jiao Tong University. He has received the B.S. degree in School of Information Science and Engineering in Southeast University, Nanjing, China, in 2012. His research interests include social network, data mining and computational intelligence.



Li Pan received the B.S. degree in electronic engineering from Xidian University, China, in 1996, and received the M.S. degree in computer science from Nanjing University of Telecommunications, China, in 1999, then received the Ph.D. degree in communication and information system from Shanghai Jiao Tong University, China, in 2002. Since 2002, he has been working in Shanghai Jiao Tong University. In 2008, he worked as a visiting scholar in IBM Research T.J. Watson Center, USA. In 2014, he worked as a visiting scholar in the computer department of Boston University, USA. His research interests focus on policy base access control, system security and social computing. He published over 60 papers, 5 co-authored books on operating system and network security, and hold 7 granted patents. In 2007 and 2008, he was awarded by Shanghai Rising-Star Program for Young Scientists in China, and Foundation for the Shanghai Talents in China respectively. In 2013, he was elected for the Shanghai Subject Chief Scientist in China.